PROGRAMMER GUIDE / 01

From Simulink to a browser-based virtual lab

Bless fuel-cell / diesel powertrain comparison | Programmer handoff | 25 October 2026

The objective

Build a user-friendly virtual experiment powered by the **same calculations as the verified MATLAB model**. Users should configure a scenario, understand energy flow, compare the powertrains and inspect the results without navigating Simulink.

27 diagrams	70 editable entries	10 reference cases
2,063 blocks and 1,727 explicit connections	52 operating/component values and 18 battery-mask settings	40 full time histories across four output streams

What you are receiving

Bless_Platform_Codex.py is the single-file, offline handoff. It contains the original model copy, its structured export, exact input/output catalogs, implementation instructions, visual requirements and MATLAB reference traces. It also checks the converted engine's results.

This guide explains the intended product, implementation order and evidence required for acceptance. The browser engine and interface are the next development task; they have not been created by this handoff.

How to give it to Codex

Suggested instruction
Prepare this handoff, read its build and visual requirements, and implement the validated browser module in my platform repository. Complete the corresponding animated visual outputs and provide the test evidence.

Recommended visual direction

Use a lightweight animated passenger van, a physical energy-flow diagram and synchronized charts. An SVG or Canvas implementation is enough for the first release. Add 3D later only if it improves learning and uses the same validated engine API.

The current study is a requested-speed energy model. Its calculations have been tested in MATLAB; its component assumptions still require calibration against measured vehicle data.

PROGRAMMER GUIDE / 02

Use the handoff

Python is the packaging and verification tool. TypeScript is the recommended browser implementation language.

Three practical steps

1. Give the programmer the Python file and access to the target platform repository. Python 3.10+ is sufficient; preparing the package requires no third-party modules or network connection.

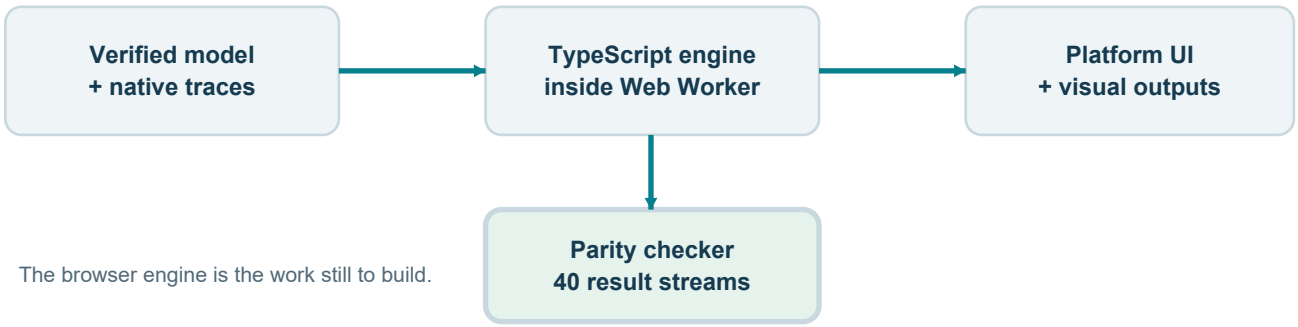
```
python Bless_Platform_Codex.py prepare --out Bless_Platform_Handoff
```

2. Have Codex read **BUILD_INSTRUCTIONS.txt**, **DESIGN_AND_ACCEPTANCE.txt** and the manifest in the extracted folder. The script rejects a nonempty destination to avoid overwriting project work.

3. Implement the engine and interface, export the reference scenarios, then run the checker. This checks every required time sample, not only the final dashboard totals.

```
python Bless_Platform_Codex.py validate-results --results-dir path/to/engine/results
```

Extracted content	Purpose
model/	Exact block graph, 70 input entries, 106 ordered output channels and 17 state blocks.
reference/	Snapshot of the verified SLX, existing model guide and extraction provenance.
validation/	Ten exact scenario definitions, 40 native CSV traces, expected metadata and previous verification evidence.
Build and visual instructions	Execution order, browser integration, required animation, chart bindings and acceptance gates.



The browser engine is the work still to build.

Additional commands: **info** summarizes the package; **prompt** prints the implementation brief; **verify-bundle** checks file hashes; **compare** checks one result stream. Run **--help** for syntax.

PROGRAMMER GUIDE / 03

Preserve the model faithfully

Numerical parity comes before visual polish. The export supplies the actual equations and state behavior.

Conversion concern	Required treatment
Clock and state	Keep the 1 s step, all 17 UnitDelay states and their initial values. Emit t=0 through the stop time, inclusive. UI playback speed cannot change the numerical clock.
Expressions and vectors	Preserve all 302 Fcn expressions, one-based indexing, precedence, units and denominator guards. Map ^ to power, not JavaScript XOR.
Battery and lookup	Retain mask validation, SOC/current/voltage/thermal restrictions, the 13-point OCV curve and its integrated stored-energy inventory.
Fuel cell and diesel	Preserve hysteresis, fuel availability, power ramps, conversion losses, diesel idle and overrun behavior.
Traceability	Map original block paths to implementation functions. Keep the engine independent of UI and resolve local From/Goto tags correctly.

Interpret the eligibility flags precisely

The native comparison flag checks distance above 0.0001 km and cumulative electric-DC and diesel unmet energy below 0.000001 kWh. It **does not directly check motor shortfall or the mass screen**. Preserve this rule for parity and show those additional limitations separately. Label it **Model comparison eligible**.

Individual refill costs require all three prices to be positive. Cost-saving comparison also requires the native comparison flag and battery depletion of at least -0.001 kWh. A larger final battery-energy surplus prevents a valid cost-saving comparison. Do not confuse individual cost availability with comparison validity.

Keep the accounting boundary visible

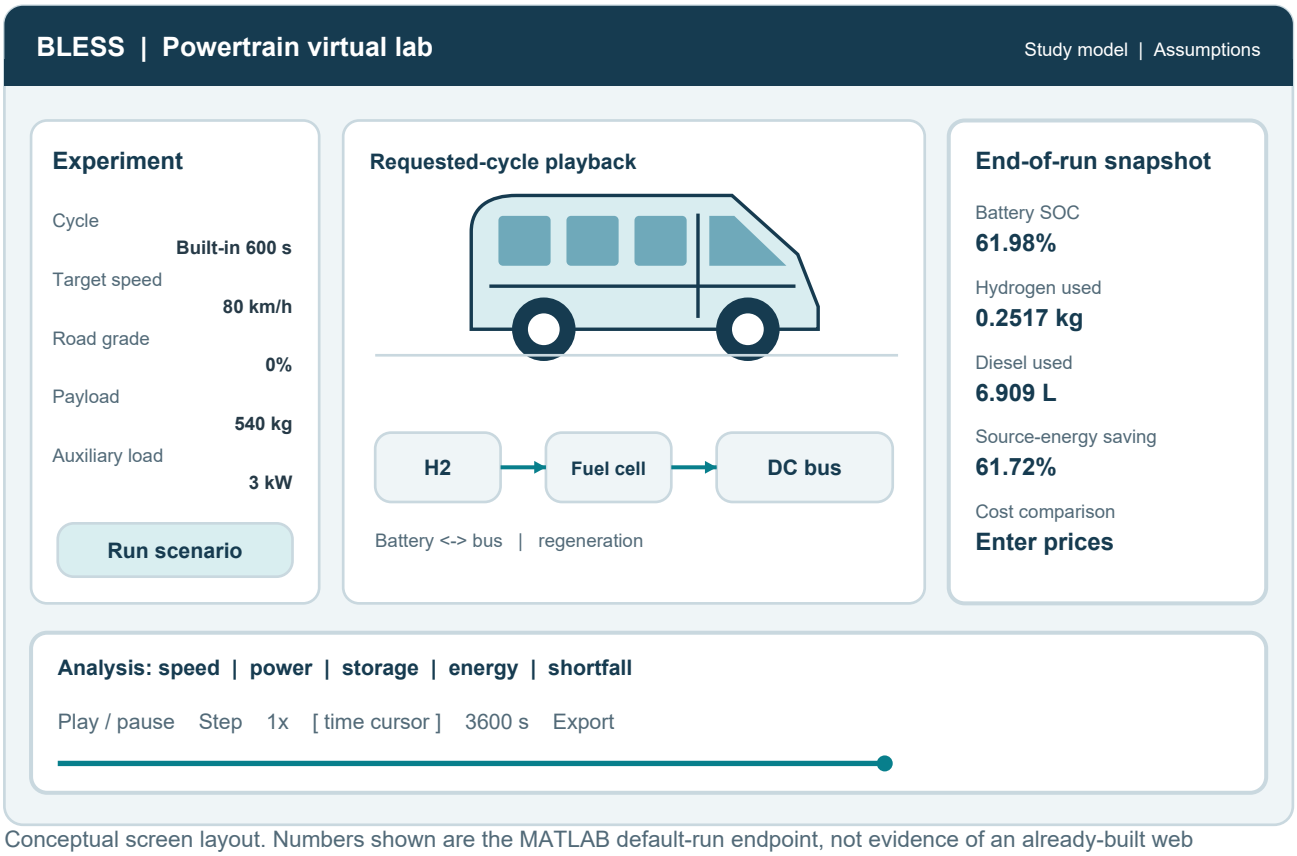
Electric source energy is hydrogen chemical energy plus **signed battery inventory depletion**. Do not add fuel-cell electrical output and gross battery discharge again. Refill cost includes hydrogen and grid recharge; it excludes purchase, maintenance and infrastructure.

The model does not predict achieved speed, detailed fuel-cell electrochemistry, tank pressure or switching transients. Default driving is a demonstration cycle, not a measured journey. No route-completion or commercial-vehicle-performance claim follows from a successful run.

PROGRAMMER GUIDE / 04

The interface to build

A clear experiment workspace, with the most useful controls and results visible first.



Conceptual screen layout. Numbers shown are the MATLAB default-run endpoint, not evidence of an already-built web application. All production values must come from the converted engine.

Area	Expected behavior
Setup	Driving cycle, grade, load, temperature, fuel-cell enable and initial storage; advanced component settings and OCV editing in an expandable panel.
Playback stage	Passenger-van motion, signed power flows, battery/hydrogen state and a persistent explanation when requested demand is unsupported.
Comparison and analysis	Electric/diesel cards, synchronized charts, raw-value table, scenario import/export and original-sample CSV/JSON export.

Interaction requirements

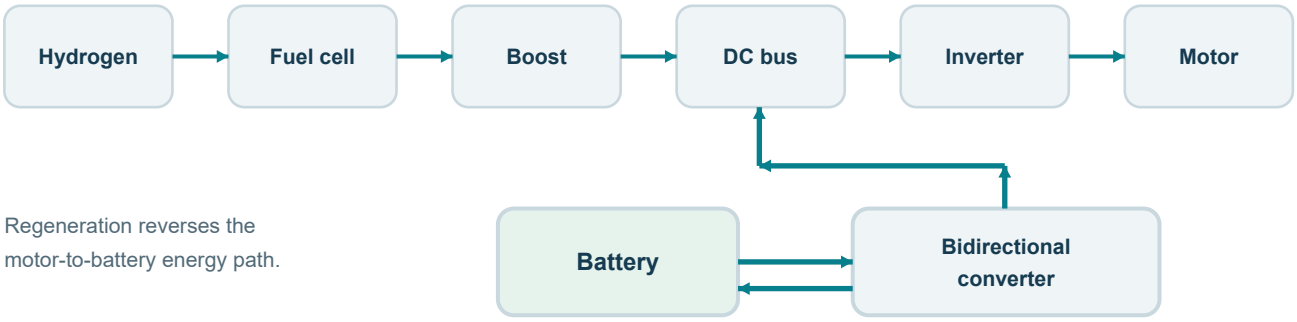
Provide run/recalculate, cancellation, play/pause, one-second step, reset, a time scrubber and playback-speed control. Edits mark old results as stale and take effect on a new run. Use a Web Worker to keep the interface responsive.

At narrow widths, stack the panels rather than shrinking labels. Support keyboard navigation, visible focus, text alongside status colors, a table alternative to charts, and reduced motion.

PROGRAMMER GUIDE / 05

Demand meaningful visual output

The moving vehicle must explain the simulation and agree with its data at every time cursor.



Regeneration reverses the motor-to-battery energy path.

Visual	Binding / acceptance
Van and road motion	Use commanded speed and the common playback cursor. Caption it Requested-cycle playback; do not call it achieved speed or proven journey completion.
Power-flow arrows	Use actual fuel-cell DC, signed battery DC and accepted regeneration. Reverse battery flow when charging. Keep the auxiliary-load path and loss indicators understandable.
Storage and limits	Use battery SOC, temperature, current and hydrogen inventory. Show fuel exhaustion, limiting behavior, motor shortfall and mass excess explicitly.
Comparison cards	Show model eligibility and cost availability. Missing prices display Enter prices; negative valid savings remain visible.
Charts	Speed; SOC and hydrogen; power sharing; battery current/temperature; source energy; shortfalls; costs only where interpretable. Keep units and legends clear.

Three essential demonstrations

Normal run: configure, calculate, watch sharing, scrub and export. **Overload:** positive shortfall appears in the stage, charts and comparison. **Fuel depletion:** hydrogen reaches zero and fuel-cell flow stops according to the result.

An additional downhill/high-SOC example should show accepted regeneration and charging limits. Animate only the accepted flow; requested braking is not automatically recovered energy.

Physical energy paths are not the same as Simulink information-signal wires. The programmer must map every animated path to a named result or verified internal telemetry.

PROGRAMMER GUIDE / 06

Verify against MATLAB

All ten native scenarios are packaged with complete comparison, electric, diesel and road traces.

Case	Duration	Changed settings
Default cycle	3,600 s	Original defaults
Battery only	3,600 s	Fuel cell disabled
Fuel-cell start	1,200 s	SOC 50%; illustrative prices
Empty hydrogen	1,200 s	SOC 50%; hydrogen 0 kg
Constant speed	1,200 s	60 km/h; constant profile
Uphill overload	600 s	20% grade
High-SOC regeneration	600 s	SOC 95%; grade -10%
Hydrogen depletion	1,200 s	SOC 50%; hydrogen 0.02 kg
Priced cycle	1,200 s	Electricity 2, H2 50, diesel 15*
Stationary	600 s	Target speed 0 km/h

* Illustrative GHS unit prices used to verify arithmetic, not current market quotations. Exact settings and parameter paths are embedded in scenarios.json.

Default 3,600 s reference endpoint

Commanded distance	Battery SOC	Hydrogen used	Diesel used
66.666667 km	61.984004%	0.251740 kg	6.908598 L

Electric source energy: **26.308696 kWh**. Diesel source energy: **68.724608 kWh**. Source-energy saving: **61.718668%**. Model comparison flag: **1**. Cost comparison remains unavailable because the default prices are zero.

What passing means

The checker compares all 40 result streams, including exact time grids and designated Boolean flags. Numeric tolerance is **1e-8 + 1e-9 x |reference|**. Inputs and source-model hash must match the result manifest. A complete default trace contains 3,601 samples.

Add unseen-input, reset/repeatability, boundary and cancellation tests. Never feed reference traces into the production engine to imitate a successful conversion. Numerical agreement establishes porting fidelity, not experimental vehicle validation.

PROGRAMMER GUIDE / 07

Implementation and handover

Keep the engine reusable so the browser interface can evolve without rewriting the model.

Stage	Reviewable deliverable
1. Extract and map	A conversion report covering every computational block, mask setting, state and output channel.
2. Build the engine	Typed scenario/state/sample APIs, deterministic execution and parameter validation; no DOM or MATLAB runtime dependency.
3. Prove parity	All ten cases and four streams pass, with a report of errors, time alignment and input/model identity.
4. Integrate the platform	A working route/component using the host design system, Web Worker, scenario controls and export contracts.
5. Complete the visuals	Synchronized vehicle/power-flow animation, accessible charts and usable desktop/phone layouts.
6. Hand over evidence	Source, setup/build instructions, integration notes, tests, result exports, screenshots and a short screen recording.

Required public engine boundary

validateScenario; createSimulation; stepSimulation; runScenario; toDisplaySample; scenario import/export; result CSV/JSON export. Use semantic field names, explicit units, schema versions and run IDs. Keep vector offsets inside the documented adapter.

Acceptance is concrete

Require desktop and phone screenshots from the real converted engine, a 20-40 second recording showing playback and a constrained/unavailable result, and chart exports with units, scenario and model version. Show the default result and an overload example. A static mockup or decorative car animation is insufficient.

Return to this conversation

Open: Fix Bless Fuel Cell MATLAB file

Codex thread ID: 01a173ef-809f-7e80-b36a-a3fb541b5c4c
Host: local. Codex can use read_thread with these identifiers and paginate older turns.

This is a local Codex link, not a public web share. A public-share request timed out. The embedded model, specifications and tests remain usable without access to this chat.

PROGRAMMER GUIDE / 08

Sources and delivery checklist

The single Python file is the portable handoff. This guide is the short explanation for its implementer.

Local source of truth

The package contains a snapshot of **Powertrain_Comparison.slx** and the existing model guide from **F:/Scripts for codex/Bless**. The source model is identified by SHA-256 in manifest.json. Preparing the package does not change it.

Source hash:
6ff353c6fd9cbb994c163e26ad183b9e6f780f45d4ac4ec788bcbc4528cf8987

Technical references for the programmer

Simulink Fcn expression rules

Discrete-time state/update semantics

Lookup-table interpolation and extrapolation

TypeScript numeric and data types

Web Worker implementation

Reduced-motion preference

Online documentation may describe newer releases. The exported R2024b block settings and native reference traces determine this conversion. This model uses UnitDelay feedback for integration; it has no DiscreteIntegrator blocks.

Final acceptance checklist

Required outcome	Evidence to request
Faithful calculations	40 native-stream comparisons pass; fresh configurations and state/reset behavior checked.
User-friendly interface	Readable controls, correct units, clear model limits, meaningful error/availability messages.
Corresponding visual output	Vehicle and energy flows, synchronized charts, comparison cards and exports from the same run.
Platform-ready code	Reusable engine API, worker integration, versioned inputs/results and documented setup.
Honest completion report	What passed, what is still limited, and no claim of deployment or measured validation without evidence.

Requested end product: a functioning browser-based virtual lab powered by the converted model, with corresponding animated and graphical results that remain correct when the scenario changes.